Gauntlet

ML-2026-01

Meridian Logistics: The Last Mile to Intelligence

This case study is based on real work Gauntlet has done with real clients. While Meridian Logistics is a fictional company, the scenario is directionally representative of what many companies are actively building today.

COMPANY BACKGROUND

Meridian Logistics is a mid-market freight brokerage headquartered in Memphis, Tennessee, with 4,200 carrier partners, $380M in revenue, and 310 employees. Half are brokers who coordinate FTL and LTL shipments for mid-size manufacturers across the continental US. The company has grown 22% year-over-year for the past three years, largely through adding broker headcount rather than improving tooling.

Meridian operates **five regional offices** (Memphis, Dallas, Atlanta, Chicago, Denver), each functioning as a semi-independent P&L. Brokers are organized by region and specialize in specific lanes and equipment types. A typical broker manages 40–60 active shipments at any given time, toggling between rate negotiation, carrier vetting, shipment tracking, and exception handling. The highest-performing brokers spend roughly 35% of their day on manual data lookup and status updates rather than revenue-generating work.

The company runs on **FreightView**, a vertical SaaS platform built in-house over six years by a small engineering team. It handles quoting, dispatch, tracking, carrier management, invoicing, and claims. It is a classic CRUD app backed by PostgreSQL with 15 tables and 40M+ rows. A broker touches

TECH STACK

Python / FastAPI
Monolith, 34 REST endpoints

React
SPA frontend, 6 core modules

Postgres
15 tables, 40M+ rows

Redis
Session cache, job queues

seven screens to manage a single shipment from quote to delivery. The two-person analytics team has a **170-request backlog** with a nine-day turnaround, mostly for ad hoc reports that brokers and office managers need to make operational decisions.

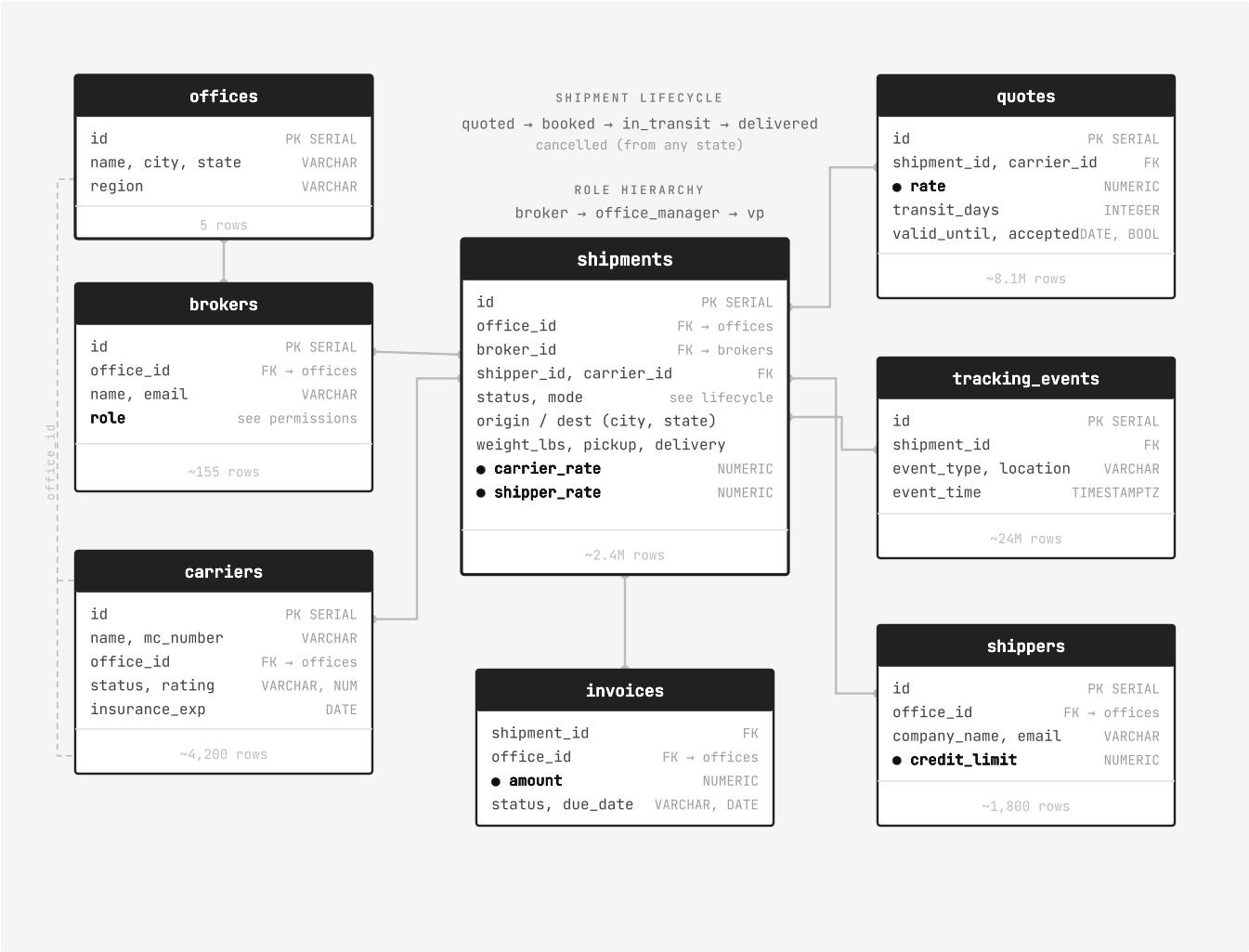
FREIGHTVIEW MODULES

MODULE	FUNCTION	DAILY ACTIVE USERS
Quote Engine	Rate lookups across carrier APIs	142
Dispatch Board	Assign loads to carriers	138
Track & Trace	Real-time shipment visibility	204
Carrier Scorecard	Performance ratings, compliance	46
Billing & Claims	Invoicing, dispute resolution	61
Analytics	Canned reports, CSV exports	89

DATA MODEL

The data model reflects Meridian's multi-office structure: every core table carries an `office_id` foreign key scoped to one of the five regional offices. Columns marked ● **SENSITIVE** contain contract rates and financial data that must be protected from both display and AI agent access.

EXHIBIT A: ENTITY RELATIONSHIP DIAGRAM



Seven additional tables (`lanes`, `carrier_lanes`, `claims`, `documents`, `notifications`, `audit_log`, `rate_history`) are documented in the full schema appendix.

PERMISSIONS MODEL

Access is scoped by **office and role**. A `broker` can only see shipments, carriers, shippers, and invoices where they are the assigned broker within their office. An `office_manager` can see all records within their office. A `vp` can see all offices. The agent must enforce these boundaries on every query it generates. A Dallas broker asking “show me all shipments” must only see Dallas shipments assigned to them, never Atlanta’s data.

WHAT BROKERS ASK

The analytics team logged the 200 most common broker requests. These six are representative:

```
READ: AGGREGATION
```

> “What’s the average transit time for LTL shipments from Dallas to Chicago over the last 90 days?”

> READ: RANKING WITH FILTER
> “Show me the top 5 carriers by on-time delivery rate for FTL loads over 20,000 lbs in the Southeast.”

> READ: MULTI-TABLE JOIN
> “Which of my shipments are currently in transit with a carrier whose insurance expires in the next 30 days?”

> WRITE: BOOKING
> “Book that Dallas–Chicago lane with Carrier #4412 at the quoted rate for next Tuesday pickup.”

> WRITE: STATUS UPDATE
> “Mark shipment #88219 as delayed and notify the shipper that the new ETA is Thursday.”

> BOUNDARY TEST: CROSS-OFFICE
> “How does our Dallas office’s on-time rate compare to Atlanta’s?” *(should succeed for a VP, fail for a broker)*

API SURFACE

FreightView exposes 34 REST endpoints. Booking a shipment transitions its status from `quoted` to `booked`, triggers a rate confirmation email to the carrier, and creates a pending invoice. Representative endpoints:

READ (18 ENDPOINTS)

`GET /shipments?broker_id=&status=`
`GET /shipments/{id}/tracking`
`GET /carriers?min_rating=&lane=`
`GET /quotes?shipment_id=`
`GET /invoices?status=overdue`
`GET /analytics/lane-performance`

WRITE (16 ENDPOINTS)

`POST /shipments (create load)`
`POST /shipments/{id}/book (assign carrier)`
`PATCH /shipments/{id}/status (update ETA)`
`POST /claims (file a claim)`
`POST /notifications/send (alert shipper)`
`DELETE /quotes/{id} (withdraw a quote)`

All endpoints require a JWT containing `broker_id`, `office_id`, and `role`. The existing API enforces row-level filtering server-side. There is no admin API exposed to the frontend.

THE PROVOCATION

> *Every screen in FreightView is just a different shaped window into the same database. What if we replaced all of them with one interface: a chat?*
>
> *CEO, Meridian Logistics*

The VP of Engineering has 90 days and four engineers to build a proof-of-concept. The goal for this phase is a working agent that handles **read queries and single-step bookings** for the Memphis office, with the architecture designed to extend to multi-office, multi-step workflows in a later phase. An initial prototype that passes the full schema and a broker question directly to an LLM generates valid SQL only **62% of the time**. Common failures include hallucinated column names, incorrect joins, and unbounded queries returning millions of rows.

ASSIGNMENT

You are the VP of Engineering. Design the architecture for a conversational agent that replaces FreightView's six-module CRUD interface with a single chat. Your architecture should be scoped to the 90-day PoC (read queries + single-step bookings, Memphis office), but designed so the five areas below can extend to full multi-office, multi-step production use.

Your proposal must address five areas:

- ☐ **Data Querying:** How does the agent translate natural language into safe, correct SQL against this schema? What prevents destructive queries, sensitive column exposure, and unbounded result sets?
- ☐ **Multi-step Workflows:** A broker says "find me the cheapest carrier and book it." That's search → quote → confirm → book, with multiple steps involving branching, failures, and human approval. How do you orchestrate that? What happens when a quote expires mid-conversation?
- ☐ **Action Execution:** The agent needs to call FreightView's 34 API endpoints, some read-only and some with financial consequences. How does the agent discover and invoke these actions? How do you enforce broker confirmation before writes execute?

- ☐ **Security & Evaluation:** How do you defend against prompt injection, cross-office data leakage, and unauthorized action escalation? A Dallas broker should never see Atlanta's data, even if they ask cleverly. How do you measure accuracy and build an evaluation harness that evolves as the agent's capabilities expand?
- ☐ **Rich Responses:** Freight data is useless as plain text. Brokers need sortable tables, status timelines, and interactive confirmation cards, all rendered inline in chat. What is the contract between agent output and the frontend? How do interactive elements trigger further actions?

Your deliverable should include:

- ☐ A **system architecture diagram** showing every component (LLM, API gateway, query engine, tool router, permission layer, frontend) and the data flows between them.
- ☐ **Interface contracts** for at least two boundaries: agent-to-database (e.g., what the generated SQL or query plan looks like before execution) and agent-to-frontend (e.g., the structured response format the chat UI renders). Use the reference payloads below as a starting point.
- ☐ A **security model** specifying where permission checks run, what gets filtered, and how you prevent bypass.
- ☐ An **evaluation plan** with concrete metrics (accuracy, latency, safety) and how you would build your initial test suite from the six example queries above.
- ☐ A **scope table** that separates what ships in the 90-day PoC from what is deferred to later phases, with brief rationale for each deferral.

Reference payloads (use these as a starting point for your contracts):

```
AGENT → DATABASE: QUERY PLAN
{
  "sql": "SELECT c.name, COUNT(*) as loads, AVG(te.delay_mins) ... ",
  "params": {"broker_id": 42, "office_id": 1},
  "max_rows": 50,
  "sensitive_columns_excluded": ["carrier_rate", "shipper_rate"],
  "requires_confirmation": false
}
```

```
AGENT → FRONTEND: STRUCTURED RESPONSE
```

```
{
  "message": "Here are your top 5 carriers for that lane.",
  "components": [
    {"type": "table", "columns": ["Carrier", "On-Time %", "Loads"], "rows":
[...]},
    {"type": "action_button", "label": "Book top carrier", "action": "POST
/shipments/book", "requires_confirmation": true}
  ]
}
```

Be specific. Your goal is to produce a concrete architecture for this system: draw out the components, define how they connect, specify the data flows, and document the contracts between services. Identify the key architectural decisions, the tradeoffs behind each one, and the failure modes you are most concerned about. This should read as a blueprint that an engineering team could pick up and start building from.